

使用 MediaPipe 在 Android 裝置上生成圖像

來源：<https://codelabs.developers.google.com/mp-image-generation-basic-android?hl=zh-tw>

步驟數：6

在這個程式碼研究室中，您將瞭解如何使用 MediaPipe Solutions，在 Android 應用程式中加入裝置端文字轉圖片生成功能。

目錄

1. [簡介](#)
2. [在 Android 應用程式中新增 MediaPipe Tasks](#)
3. [設定圖片產生器](#)
4. [整合應用程式](#)
5. [部署及測試應用程式](#)
6. [恭喜！](#)

1. 簡介

什麼是 MediaPipe ?

[MediaPipe Solutions](#) 可讓您在應用程式中套用機器學習 (ML) 解決方案。這個框架可讓您設定預先建構的處理管道，為使用者提供即時、吸引人且實用的輸出內容。您甚至可以使用 [MediaPipe Model Maker](#) 自訂許多解決方案，更新預設模型。

文字轉圖像生成是 MediaPipe Solutions 提供的其中一項機器學習工作。

雖然 MediaPipe Tasks 適用於 Android、iOS、Python (包括 Raspberry Pi!) 和網頁，但這項實驗性工作目前僅適用於 Android。此外，建議您在至少有 8 GB RAM 的裝置上進行測試。

在本程式碼研究室中，您會從幾乎空白的 Android 應用程式開始，逐步完成多個步驟，直到能夠直接在 Android 裝置上生成新圖片。

課程內容

- 如何使用 [MediaPipe Tasks](#)，在 Android 應用程式中實作在本機執行的文字轉圖像生成功能。

軟硬體需求

- 已安裝 [Android Studio](#) 版本 (本程式碼研究室是使用 Android Studio Giraffe 編寫及測試)。
 - Android 裝置的 RAM 達 8 GB 以上。
 - 具備 Android 開發的基本知識，且能夠執行預先編寫的 Python 指令碼。
-

2. 在 Android 應用程式中新增 MediaPipe Tasks

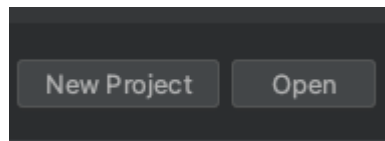
下載 Android 範例應用程式

本程式碼研究室會從預先製作的範例開始，其中包含用於基本圖像生成版本的 UI。您可以在官方 MediaPipe 範例存放區 ([這裡](#)) 找到起始應用程式。按一下「Code」>「Download ZIP」，複製存放區或下載 zip 檔案。

- 封存檔包含與本程式碼研究室無關的其他 MediaPipe 範例。本程式碼研究室的程式碼位於 `codelabs/image_generation_basic/android` 資料夾中。
- 我們已為本程式碼研究室實作基本 UI 和樣板程式碼。原始碼位於 `codelabs/image_generation_basic/android/start/` 資料夾中。
- 在本教學課程的後續步驟中，您將編寫程式碼來使用一組二進位檔。
- 如果您不知道如何進行任何步驟，可以查看 `codelabs/image_generation_basic/android/finish/` 資料夾中的完成應用程式。

將應用程式匯入 Android Studio

1. 開啟 Android Studio。
2. 在「Welcome to Android Studio」畫面中，選取右上角的「Open」。



3. 前往複製或下載存放區的位置，然後開啟 `codelabs/image_generation_basic/android/start` 目錄。
4. 此時，應用程式不應編譯，因為您尚未納入 MediaPipe Tasks 依附元件。

您將前往 **build.gradle** 檔案，向下捲動至「// Step 1 - Add dependency」，修正應用程式並讓其執行。然後加入下列程式碼行，並按下 Android Studio 頂端橫幅中顯示的「Sync Now」按鈕。

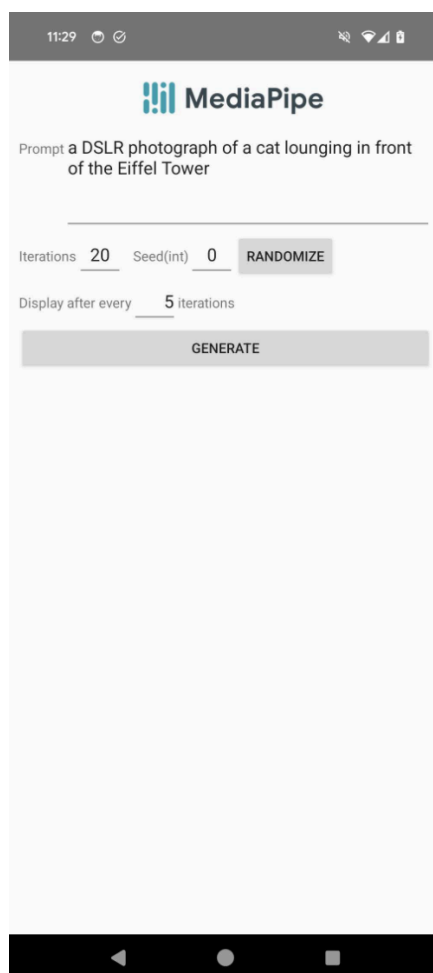
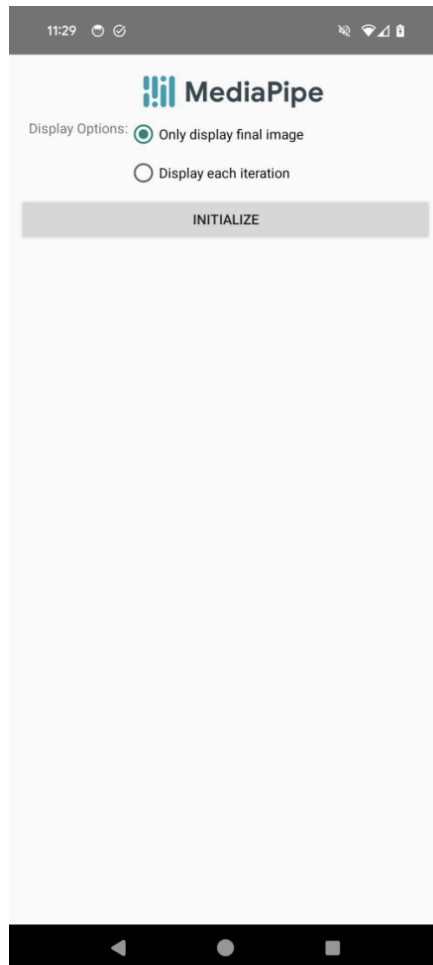


```
// Step 1 - Add dependency
implementation 'com.google.mediacpipe:tasks-vision-image-generator:latest.release'
```

同步完成後，請點選 Android Studio 右上方的綠色「執行」箭頭 (



)，確認所有項目都已正確開啟及安裝。應用程式應該會開啟畫面，當中包含兩個圓形按鈕和標示為「INITIALIZE」的按鈕。點選該按鈕後，系統應會立即帶您前往另一個使用者介面，當中包含文字提示詞和其他選項，以及標示為「生成」的按鈕。



很抱歉，這就是範例應用程式的全部內容，現在請學習如何完成這個應用程式，並開始在裝置上生成新圖片！

3. 設定圖片產生器

在這個範例中，大部分的圖像生成工作都會在 **ImageGenerationHelper.kt** 檔案中進行。開啟這個檔案後，您會發現類別頂端附近有個名為 `imageGenerator` 的變數。這是 `Task` 物件，將在圖像生成應用程式中執行繁重的工作。

在該物件下方，您會看到名為 `initializeImageGenerator()` 的函式，以及下列註解：`// Step 2 - initialize the image generator`。如您所料，您將在此初始化 **ImageGenerator** 物件。將該函式主體替換為下列程式碼，設定圖像生成模型路徑並初始化 **ImageGenerator** 物件：

```
// Step 2 - initialize the image generator
val options = ImageGeneratorOptions.builder()
    .setImageGeneratorModelDirectory(modelPath)
    .build()

imageGenerator = ImageGenerator.createFromOptions(context, options)
```

下方會顯示另一個名為 `setInput()` 的函式。這個函式會接受三個參數：用於定義生成圖片的 **prompt** 字串、`Task` 在生成新圖片時應經歷的疊代次數，以及 **seed** 值。這個值可用於根據相同提示建立圖片的新版本，並在採用相同 **seed** 時生成相同圖片。當您嘗試建立會顯示中間步驟的圖片時，這個函式的用途是為圖像生成器設定這些初始參數。

請繼續將 `setInput()` 主體 (您會看到 `// Step 3 - accept inputs` 註解) 替換為這行程式碼：

```
// Step 3 - accept inputs
imageGenerator.setInput(prompt, iteration, seed)
```

接下來的兩個步驟會進行生成作業。`generate()` 函式接受與 `setInput` 相同的輸入內容，但會建立圖片做為單次呼叫，不會傳回任何中間步驟圖片。您可以將這個函式的主體 (包括註解 `// Step 4 - generate without showing iterations`) 替換為下列內容：

```
// Step 4 - generate without showing iterations
val result = imageGenerator.generate(prompt, iteration, seed)
val bitmap = BitmapExtractor.extract(result?.generatedImage())
return bitmap
```

請務必瞭解這項工作是同步進行，因此您需要從背景執行緒呼叫函式。您將在本程式碼研究室稍後進一步瞭解這項功能。

您將在這個檔案中執行的最後一個步驟，是填入 `execute()` 函式 (標示為步驟 5)。這會接受參數，告知是否應傳回 **ImageGenerator** `execute()` 函式執行的生成單一步驟的中間圖片。將函式主體替換為下列程式碼：

```
// Step 5 - generate with iterations
val result = imageGenerator.execute(showResult)

if (result == null || result.generatedImage() == null) {
    return Bitmap.createBitmap(512, 512, Bitmap.Config.ARGB_8888)
        .apply {
            val canvas = Canvas(this)
            val paint = Paint()
            paint.color = Color.WHITE
            canvas.drawPaint(paint)
        }
}

val bitmap =
    BitmapExtractor.extract(result.generatedImage())

return bitmap
```

輔助檔案的內容就是這樣。在下一節中，您將填寫 ViewModel 檔案，處理這個範例的邏輯。

4. 整合應用程式

MainViewModel 檔案會處理 UI 狀態，以及與這個範例應用程式相關的其他邏輯。請立即開啟這個檔案。

檔案頂端應該會顯示註解「// Step 6 - set model path」。您可以在這裡告知應用程式，生成圖片所需的模型檔案位於何處。在本範例中，您會將值設為 `/data/local/tmp/image_generator/bins/`。

```
// Step 6 - set model path
private val MODEL_PATH = "/data/local/tmp/image_generator/bins/"
```

然後向下捲動至 `generateImage()` 函式。您會在函式底部看到步驟 7 和步驟 8，分別用於產生傳回的疊代項目或無疊代項目的圖片。由於這兩項作業都是*同步*進行，您會發現這些作業都包裝在協同程式中。首先，請使用這段程式碼取代「// Step 7 - Generate without showing iterations」，從 **ImageGenerationHelper** 檔案呼叫 `generate()`，然後更新 UI 狀態。

```
// Step 7 - Generate without showing iterations
val result = helper?.generate(prompt, iteration, seed)
_uiState.update {
    it.copy(outputBitmap = result)
}
```

步驟 8 稍微複雜一點。由於 `execute()` 函式只會執行一個步驟，而不是圖像生成*所有*步驟，因此您需要透過迴圈個別呼叫每個步驟。您還需要判斷是否應向使用者顯示目前步驟。最後，如果應顯示目前疊代，您將更新 UI 狀態。你現在可以執行上述所有操作。

```
// Step 8 - Generate with showing iterations
helper?.setInput(prompt, iteration, seed)
for (step in 0 until iteration) {
    isDisplayStep =
        (displayIteration > 0 && ((step + 1) % displayIteration == 0))
    val result = helper?.execute(isDisplayStep)

    if (isDisplayStep) {
        _uiState.update {
            it.copy(
                outputBitmap = result,
                generatingMessage = "Generating... (${step + 1}/${iteration})",
            )
        }
    }
}
```

此時，您*應該*可以安裝應用程式、初始化圖像生成器，然後根據文字提示詞建立新圖片

... except now the app crashes when you try to initialize the image generator. 發生這種情況的原因是，您需要將模型檔案複製到裝置。如要取得已知可用的第三方模型最新資訊、轉換模型以用於這項 MediaPipe 工作，以及將模型複製到裝置，請參閱官方文件的[這個部分](#)。

除了直接將檔案複製到開發裝置，您也可以設定 Firebase 儲存，在執行階段將必要檔案直接下載到使用者的裝置。

5. 部署及測試應用程式

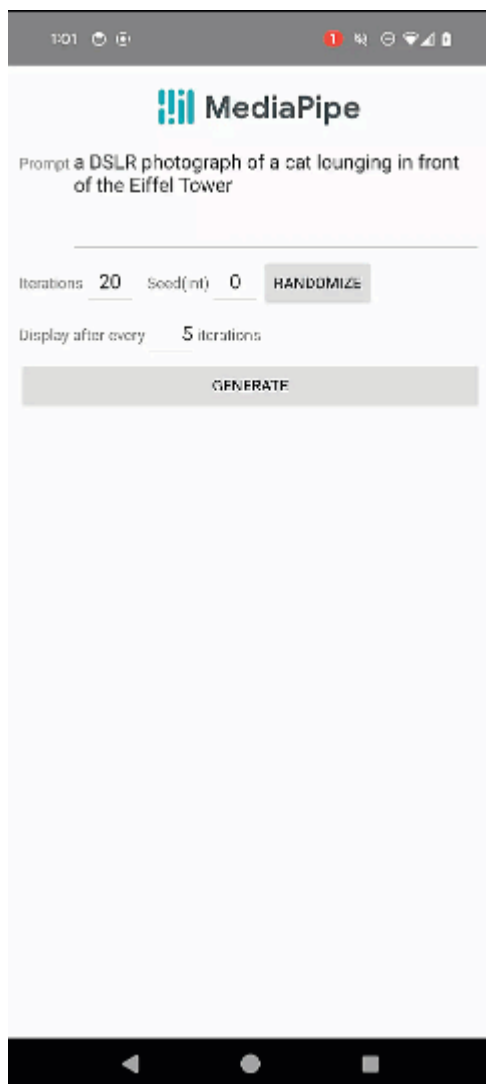
完成上述步驟後，您應該就能使用應用程式，接受文字提示並完全在裝置端生成新圖片！請將應用程式部署到實體 Android 裝置進行測試，但請注意，您必須使用記憶體至少 8 GB 的裝置。

1. 在 Android Studio 工具列中，按一下「Run」圖示 (



) 執行應用程式。

2. 選取生成步驟類型 (最終版或疊代版)，然後按下「初始化」按鈕。
3. 在下一個畫面中，設定所需屬性，然後按一下「生成」按鈕，即可查看工具生成的內容。



步驟 6 · 約 0 分鐘

6. 恭喜！

你成功了！在本程式碼研究室中，您已學會如何在 Android 應用程式中新增裝置端文字轉圖像生成功能。

後續步驟

圖像生成工作還有更多用途，包括：

- 使用基礎圖片透過外掛程式建構生成的圖片，或透過 Vertex AI 訓練自己的額外 LoRA 權重。
- 使用 Firebase 儲存空間在裝置上擷取模型檔案，不必使用 ADB 工具。

我們很期待看到您運用這項實驗性工作製作的各種酷炫內容，也請密切關注 MediaPipe 團隊推出的更多 Codelabs 和內容！
